Adebola Elisha

AI Engineer

+2347049156715 | github.com/garvery17 | portfolioadebola.netlify.app | linkedin.com/in/adebola-elisha-77a1a8257

Education & Certifications

Federal University of Technology 2022 – 2026
B.Eng. Mechatronics and Artificial Inteligence

Certifications: Machine Learning – Kaggle, 2025 · Advanced Data Science – NCAIR, 2026

Technical Skills

Languages: Python, TypeScript/JavaScript, SQL
AI / LLM Engineering: RAG (hybrid dense + sparse retrieval, RRF fusion), LangGraph multi-agent orchestration, OpenAI GPT-4o / GPT-4o-mini, Whisper (STT), streaming TTS, prompt engineering, embeddings, evaluation/critic loops
Backend & APIs: FastAPI, Pydantic, REST API design, async Python, Uvicorn
Data & Vector Stores: Pinecone, Qdrant, PostgreSQL, BM25 / hybrid search
Cloud & DevOps: AWS (ECS Fargate, ALB, EC2, S3, ECR, IAM, CloudWatch), Docker & Docker Compose, CI/CD deployment workflows
Frontend & Tooling: Next.js, React, Playwright, Chrome Extensions (Manifest V3), Git/GitHub

Projects

nobaze — Production RAG Knowledge Base nobaze.vercel.app/

- **Context:** Most RAG demos stop at "it retrieves and answers" – I set out to build one that behaved like a real product: accurate retrieval, low-latency voice interaction, and a system someone could actually run in production.
- **Action:** Combined dense vector search (Pinecone, `text-embedding-3-small`) with BM25 keyword search fused via Reciprocal Rank Fusion, so the system doesn't miss exact-term matches that pure semantic search glosses over; paired `gpt-4o-mini` for generation to keep inference cost low without sacrificing answer quality; added Whisper STT and sentence-buffered streaming TTS so voice responses start speaking before the full answer is generated, cutting perceived latency.
- **Result:** Shipped a fully working ingest → hybrid-search → generate → voice pipeline (5 REST endpoints) containerized with Docker Compose and deployed on EC2; BM25 index rebuilds from Postgres on startup, avoiding a dependency on external search infrastructure and keeping cold-starts fast.

Ingenious Agentic — Multi-Agent Research System *https://agentic-ingenious.vercel.app/*

- **Context:** Single-prompt LLM reports are fast but unreliable – no fact-checking, no revision, no way to catch a weak first draft. I wanted a system that researched and self-corrected the way a small analyst team would.
- **Action:** Designed a 5-agent LangGraph workflow (Planner → Researcher → Analyst → Writer → Critic) where the Critic scores each report and routes it back to the Planner for another pass if it scores below a 7.0 threshold, turning report quality into a measurable, enforced gate rather than a hope; added Qdrant as persistent semantic memory so past research improves future queries instead of starting from zero every run.
- **Result:** Deployed the backend on AWS ECS Fargate behind an ALB with secrets injected from S3, and the Next.js frontend on Vercel; diagnosed and fixed an HTTPS/HTTP mixed-content failure between them using Vercel rewrites as a proxy, and resolved Pydantic startup crashes caused by quoted `.env` values – the kind of deployment friction that separates a working demo from a working service.

Docs to Docs — Documentation-to-Guide Pipeline *https://agentic-ingenious.vercel.app/*

- **Context:** Engineers lose real time hunting through unfamiliar documentation sites just to get started with a new tool. I wanted a one-click way to turn any docs site into a single, focused onboarding guide.
- **Action:** Built a 4-stage pipeline – a Playwright crawler that scores discovered URLs by relevance keywords (`install`, `quickstart`, `features`) instead of crawling blindly; an LLM classifier that discards irrelevant pages before they reach generation, keeping cost and noise down; a Writer stage with a critic/revise loop that checks each section for code blocks, numbered steps, and usage examples before accepting it, so quality is enforced rather than assumed.
- **Result:** Delivers a formatted `.docx` guide via a Chrome extension in 2–5 minutes, backed by FastAPI on ECS Fargate behind an ALB with output served through time-limited S3 presigned URLs; solved Windows-specific `asyncio` event-loop and Playwright-in-thread-pool issues that were blocking reliable concurrent crawls.